

Informe de Laboratorio 02

Tema: Revisión de elementos de programación

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón schirinosne@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
02	Revisión de elementos de programación	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 29 Mayo 2023	Al 05 Junio 2023

1. Tarea

- Invertir un matriz de enteros (2 pts)
- Rotación a la Izquierda (3 pts)
- Triángulo recursivo (4 pts)

2. Equipos, materiales y temas utilizados

- Realizar ejercicios en temas de Estructuras de datos, tipos de datos abstractos, bucles, Arrays, Listas enlazadas, Recursión
- Java
- TAD
- Arrays
- C.m. Construye responsablemente soluciones haciendo uso de estructuras de datos y algoritmos, siguiendo un proceso adecuado para resolver problemas computacionales que se ajustan al uso de los recursos disponibles y a especificaciones concretas.

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- <https://github.com/BastleyNait/EDA-LAB-B-23A.git>
- URL para el laboratorio 04 en el Repositorio GitHub.
- <https://github.com/BastleyNait/EDA-LAB-B-23A/tree/main/lab02>

4. Ejercicios designados

4.1. Ejercicio 1

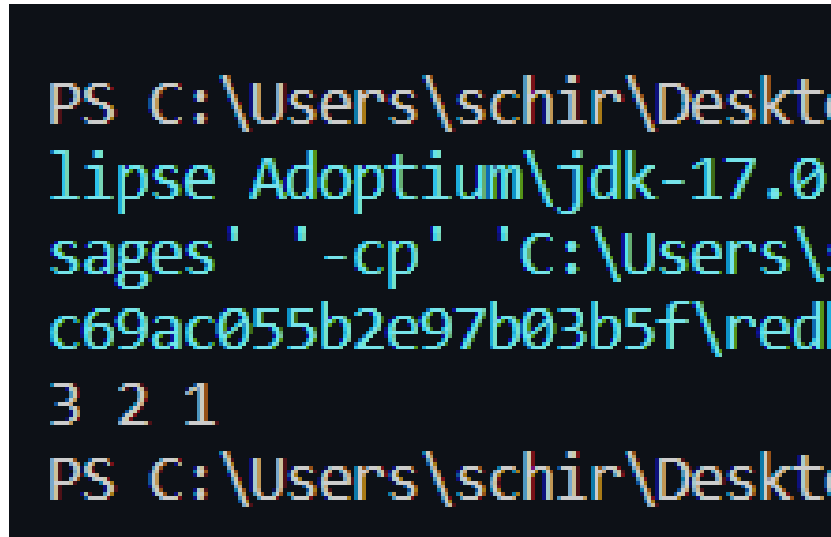
- El programa crea un array, invierte sus elementos y luego los imprime en la consola.

Listing 1: Ejercicio01.java

```
1 public class Ejercicio01 {
2     public static void main(String args[]) {
3         int [] A = {1,2,3}; // Creamos un array A con elementos 1, 2 y 3
4         int [] B = invertirArray(A); // Llamamos al mtodo invertirArray y pasamos el array A
5             como argumento
6         imprimir(B); // Llamamos al mtodo imprimir y pasamos el array invertido B como
7             argumento
8     }
9
10    // Mtodo para invertir un array
11    public static int[] invertirArray(int[] A){
12        int [] Ain = new int[A.length]; // Creamos un nuevo array Ain con la misma longitud
13        que el array A
14        for (int i = 0, j = A.length - 1; i < Ain.length; i++, j--) {
15            Ain[j] = A[i]; // Asignamos el elemento en la posicin i del array A al elemento en
16            la posicin j del array Ain
17        }
18        return Ain; // Devolvemos el array invertido Ain
19    }
20
21    // Mtodo para imprimir un array
22    public static void imprimir(int [] a) {
23        for (int i = 0; i < a.length; i++) {
24            System.out.print(a[i] + " "); // Imprimimos cada elemento del array separado por
25            un espacio
26        }
27        System.out.println(); // Imprimimos una nueva lnea despues de imprimir el array
28        completo
29    }
30 }
```

- Crea un array llamado A con elementos 1, 2 y 3.
- Llama al método invertirArray pasando el array A como argumento y almacena el resultado en un nuevo array llamado B.
- Llama al método imprimir pasando el array invertido B como argumento.

- El método `invertirArray` recibe un array `A` como argumento, crea un nuevo array llamado `Ain` con la misma longitud que `A` y realiza un bucle para invertir los elementos de `A` en `Ain`, asignando el elemento en la posición `i` de `A` al elemento en la posición `j` de `Ain`.
- El método `imprimir` recibe un array `a` como argumento y realiza un bucle para imprimir cada elemento del array separado por un espacio. Luego imprime una nueva línea.
- Ejecucion del codigo:



```
PS C:\Users\schir\Desktop
lipse Adoptium\jdk-17.0
sages' '-cp' 'C:\Users\s
c69ac055b2e97b03b5f\redl
3 2 1
PS C:\Users\schir\Desktop
```

4.2. Ejercicio 2

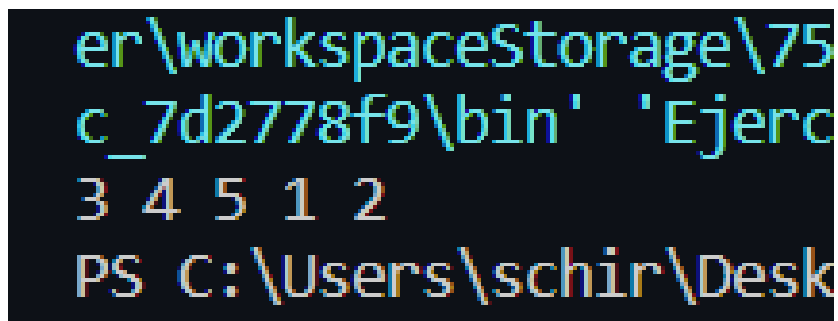
- el programa crea un array, realiza una rotación hacia la izquierda de los elementos del array y luego los imprime en la consola.

Listing 2: Ejercicio02.java

```
1 public class Ejercicio02 {
2     public static void main(String[] args) {
3         int [] A = {1,2,3,4,5}; // Creamos un array A con elementos 1, 2, 3, 4 y 5
4         int [] B = rotarIzquierdaArray(A, 2); // Llamamos al mtodo rotarIzquierdaArray y
5             pasamos el array A y el valor 2 como argumentos
6         imprimir(B); // Llamamos al mtodo imprimir y pasamos el array rotado B como argumento
7     }
8
9     // Mtodo para rotar un array hacia la izquierda
10    public static int[] rotarIzquierdaArray(int[] A, int d){
11        int[] Aiz = new int[A.length]; // Creamos un nuevo array Aiz con la misma longitud
12            que el array A
13        for (int i = 0; i < A.length; i++) {
14            int j = (i + (A.length - d)) % A.length; // Calculamos la nueva posicin j para el
15                elemento en la posicin i despues de la rotacin
16            Aiz[j] = A[i]; // Asignamos el elemento en la posicin i del array A al elemento en
17                la posicin j del array Aiz
18        }
19        return Aiz; // Devolvemos el array rotado Aiz
20    }
```

```
16 }
17
18 // Metodo para imprimir un array
19 public static void imprimir(int [] a) {
20     for (int i = 0; i < a.length; i++) {
21         System.out.print(a[i] + " "); // Imprimimos cada elemento del array separado por
                un espacio
22     }
23     System.out.println(); // Imprimimos una nueva linea despues de imprimir el array
                completo
24 }
25 }
```

- Crea un array llamado A con elementos 1, 2, 3, 4 y 5.
- Llama al método `rotarIzquierdaArray` pasando el array A y el valor 2 como argumentos, y almacena el resultado en un nuevo array llamado B.
- Llama al método `imprimir` pasando el array rotado B como argumento.
- El método `rotarIzquierdaArray` recibe un array A y un valor d como argumentos, crea un nuevo array llamado Aiz con la misma longitud que A, y realiza un bucle para rotar los elementos de A hacia la izquierda. Calcula la nueva posición j para el elemento en la posición i después de la rotación y asigna el elemento correspondiente de A al elemento en la posición j de Aiz.
- El método `imprimir` recibe un array a como argumento y realiza un bucle para imprimir cada elemento del array separado por un espacio. Luego imprime una nueva línea.
- Ejecucion del codigo:



4.3. Ejercicio 3

- El programa utiliza la recursión para imprimir un triángulo de asteriscos con la cantidad de filas especificada.

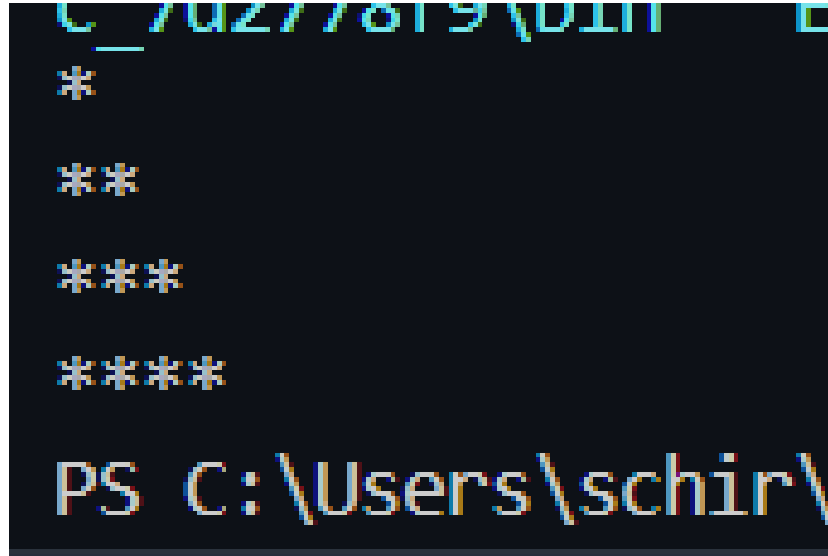
Listing 3: Ejercicio03.java

```
1 public class Ejercicio03 {
2     static int contador = 1; // Variable esttica para contar el nmero de asteriscos en cada
        nivel del tringulo
3
4     public static void main(String[] args) {
```

```
5      trianguloRecursivo(4); // Llamamos al mtodo trianguloRecursivo con el valor de base
        igual a 4
6    }
7
8    public static void trianguloRecursivo(int base){
9        // Si la base llega a ser 0, no hay nada ms que imprimir y se termina la recursin
10       if(base == 0) {
11           return;
12       } else {
13           // Imprimimos asteriscos en cada nivel del tringulo
14           for (int i = 0; i < contador; i++) {
15               System.out.print("*");
16           }
17           System.out.println(); // Imprimimos una nueva lnea despus de imprimir los
               asteriscos del nivel actual
18
19           contador++; // Incrementamos el contador para el siguiente nivel del tringulo
20           trianguloRecursivo(base - 1); // Llamamos de forma recursiva al mtodo,
               disminuyendo la base en 1 en cada llamada
21       }
22   }
23 }
```

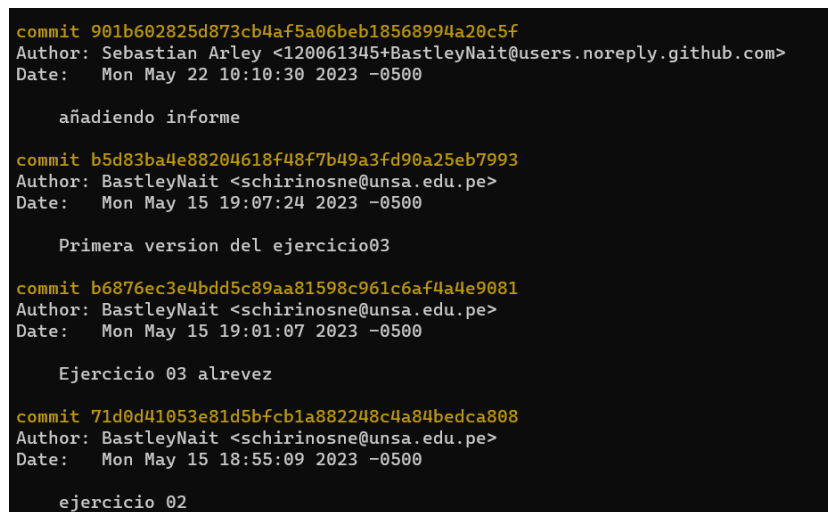
- El método trianguloRecursivo recibe un parámetro base que representa la cantidad de filas del triángulo. Si la base llega a ser 0, la recursión se detiene y no hay nada más que imprimir. En caso contrario, se realiza lo siguiente:
- Se realiza un bucle para imprimir asteriscos en cada nivel del triángulo. La cantidad de asteriscos en cada nivel está determinada por la variable contador.
- Después de imprimir los asteriscos del nivel actual, se imprime una nueva línea.
- Se incrementa el valor de contador para el siguiente nivel del triángulo.
- Se realiza una llamada recursiva al método trianguloRecursivo, disminuyendo la base en 1 en cada llamada.
- La recursión se repite hasta que la base llegue a ser 0 y se complete la impresión del triángulo de asteriscos.

- Ejecucion del codigo:



4.4. Commits del trabajo

- Acá estan algunos de los commits mas importantes en este trabajo:



4.5. Estructura de laboratorio 01

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab02/
| Ejercicio01.java
| Ejercicio02.java
| Ejercicio03.java
| lab02.iml
```

```
|  
\---latex  
|   EDA_lab02_schirinosne.pdf  
|   EDA_lab02_schirinosne.tex  
|  
+---img  
|   Commits.png  
|   Ejercicio01.png  
|   Ejercicio02.png  
|   Ejercicio03.png  
|   logo_abet.png  
|   logo_episunsa.png  
|   logo_unsa.jpg  
|  
\---src  
|   Ejercicio01.java  
|   Ejercicio02.java  
|   Ejercicio03.java
```

5. Pregunta: ¿Qué diferencia hay entre un List y un ArrayList en Java?

- La diferencia entre un List y un ArrayList en Java se encuentra en la implementación y en las características específicas de cada una:
- List es una interfaz en Java que define el comportamiento general de una lista, incluyendo operaciones como agregar elementos, acceder a elementos por índice, eliminar elementos y otras funcionalidades comunes. Es una interfaz de alto nivel que no especifica la implementación subyacente.
- ArrayList, por otro lado, es una implementación específica de la interfaz List. Representa una lista basada en un array dinámico, lo que significa que internamente utiliza un array para almacenar los elementos y se redimensiona automáticamente según sea necesario. Proporciona un acceso rápido a los elementos por índice y es eficiente para operaciones de lectura y escritura. Sin embargo, la inserción o eliminación de elementos en posiciones intermedias puede ser costosa en términos de rendimiento, ya que puede requerir desplazar los elementos.

6. Pregunta: ¿Qué beneficios y oportunidades ofrecen las clases genéricas en Java?

- En cuanto a las clases genéricas en Java, ofrecen beneficios y oportunidades significativas:
- Permiten la creación de clases y métodos que pueden ser parametrizados con tipos específicos. Esto proporciona un alto grado de reutilización de código, ya que una clase o método genérico puede ser utilizado con diferentes tipos de datos sin necesidad de escribir implementaciones específicas para cada tipo.
- Ayudan a garantizar la seguridad de tipos en tiempo de compilación. Al especificar el tipo de datos que se utilizará en una clase genérica, el compilador de Java puede detectar posibles errores de tipos antes de la ejecución del programa, lo que ayuda a evitar errores comunes y a mejorar la robustez del código.

- Mejoran la legibilidad y mantenibilidad del código. Al utilizar clases genéricas, se puede expresar de manera más clara y concisa la intención y el propósito del código. Además, al utilizar tipos genéricos, se evita la necesidad de conversiones explícitas entre tipos en muchas situaciones.
- Permiten la creación de estructuras de datos genéricas y algoritmos reutilizables. Las clases genéricas son especialmente útiles para implementar estructuras de datos como listas, pilas, colas, árboles, etc., que pueden manejar diferentes tipos de datos de manera flexible y segura.

7. Rúbricas

7.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

7.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

Nivel				
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	x	2	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	x	4	
Total		20		18	

8. Referencias

- <https://www.w3schools.com/java/>
- <https://docs.oracle.com/javase/7/docs/api/java/util/List.html>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>